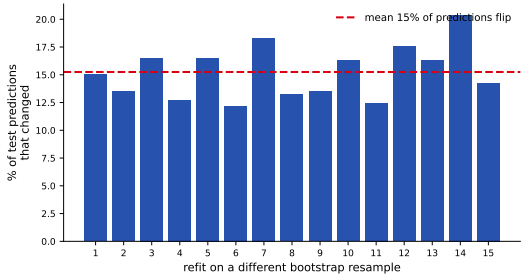
One tree is twitchy

A single decision tree ([17]) is **high variance**: refit it on a slightly different sample of the same data and it can change a lot.

Predict-first: retrain an unrestricted Titanic tree on 15 bootstrap resamples. What fraction of the *test* predictions change?



About **15%** flip – roughly one passenger in seven gets a different verdict just from re-sampling the training rows. Same method, same data size, different answers.

The plan: **average many trees** so the noise cancels. That is *bagging*, then *random forests*.

Outline

Bagging: averaging many trees

From bagging to Random Forest

Random Forest in practice

Bagging: averaging many trees

Train many trees on resampled data, then average the noise away.

Bagging = Bootstrap AGGregating

One recipe to turn a high-variance model into a stable one:

1. **Bootstrap**: draw many resampled training sets.
2. Train one model on each.
3. **Aggregate**: combine their predictions (average / vote).

Bagging is **general** – it works for any model – but it helps most for **unstable, high-variance** learners, so **deep trees** are the perfect ingredient.

A **Random Forest** is bagging *specialized to trees* with one extra twist (Section 2). For now: just average many trees.

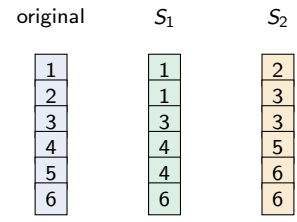
Bootstrap: resample the rows, with replacement

Reminder. A **bootstrap sample** draws rows **with replacement** (recall it from resampling / cross-validation).

In general, draw m rows from your n training rows; the classic bootstrap takes $m = n$.
Each sample comes out *slightly different*:

- some rows appear **twice or more**, others are **left out**;
- the **row mix changes** $\Rightarrow$ different counts $\Rightarrow$ different splits $\Rightarrow$ a different tree.

You can also resample **columns** (features), not just rows – that idea returns as Random Forest's per-split feature subset (Section 2).



Two bootstrap samples: repeats and gaps.

Aggregate: vote or average

Classification (Titanic): each tree votes survived / died; take the **majority** – or average the predicted probabilities ("soft vote").

Regression (Yerevan rent): each tree predicts a number; take the **mean** across trees.

Many noisy-but-roughly-right trees, averaged, give a steadier prediction than any single one of them.

By hand: three bootstrap samples, then aggregate

Six training rows, IDs 1–6. Draw **three** bootstrap samples (with replacement); each trains one tree. The rows left out are **out-of-bag (OOB)**:

sample	in-bag (rows drawn)	OOB (left out)
S_1	2, 2, 3, 5, 5, 6	1, 4
S_2	1, 1, 3, 4, 4, 6	2, 5
S_3	2, 3, 3, 5, 6, 6	1, 4

Each sample leaves a couple of rows out – we use that next. Now **aggregate** the three trees on a new passenger (their predictions are *given*, not fit by hand):

$$\{\text{survived},\ \text{died},\ \text{survived}\} \ \Rightarrow\ \textbf{majority} = \text{survived}.$$

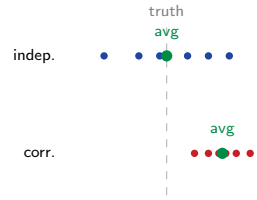
(Regression: if they said 240, 300, 270 kAMD $\rightarrow$ mean $= 270$.) With only 6 rows the out-fraction jumps around; the stable $\sim$37% is a large-n limit (we use it for OOB shortly).

Why averaging helps – and why trees

Averaging cancels **independent** errors but not **shared** ones.

- Trees that err in *different* directions → errors cancel.
- Trees that make the *same* mistake → averaging keeps it.

The naive hope (callback: cross-validation): if the trees were *independent*, M of them would cut variance by $1/M$ – like CV's $1/k$. They are **not** independent; the next frame quantifies how much that hurts.



Independent: avg lands on truth.
Correlated: avg stays off.

Why trees? Bagging only helps **high-variance** learners. Bagging a stable model (e.g. linear regression) barely moves it – little variance to average away. That is why we bag **deep** (high-variance) trees.

Predict-first: can averaging fix a biased model?

Suppose every tree **underfits** in the same way – all too shallow, all leaning toward the same wrong answer. Will averaging many of them fix it?

Predict-first: can averaging fix a biased model?

Suppose every tree **underfits** in the same way – all too shallow, all leaning toward the same wrong answer. Will averaging many of them fix it?

No. Averaging copies of a biased model returns the same biased answer. Averaging attacks **variance** (the scatter), not **bias** (the systematic offset).

Bagging lowers **variance** and leaves **bias** about unchanged – which is why the formula on the next frame has *no bias term*, and why we bag *low-bias, high-variance* deep trees.

Briefly: why averaging hits a floor

We will not dwell on the algebra. Averaging M trees, each with variance σ^2 and **average pairwise correlation** ρ, gives (ESL):

$$\mathrm{Var}(\bar{f}) = \underbrace{\rho\,\sigma^2}_{\text{the floor}} + \underbrace{\frac{1-\rho}{M}\,\sigma^2}_{\text{averaging kills this}} .$$

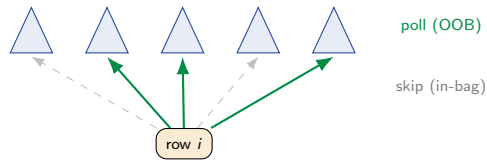
- More trees ($M \uparrow$) kill the second term – but not the first.
- The **floor** $\rho\sigma^2$ is set by how *correlated* the trees are.

The point (not the formula): more trees help up to a point; to go further we must make the trees **less alike** – the next section.

Out-of-bag (OOB) error: free validation

Each bootstrap leaves about **37%** of rows out ($(1-\frac{1}{n})^n \to \frac{1}{e}$). A row is **out-of-bag** for the trees that never saw it – so score it using **only those trees**.

- Repeat for every row → an honest held-out estimate,
- **no separate validation split needed** for tuning,
- just set `oob_score=True`.



OOB error is **comparable to a held-out / CV estimate**, usually slightly *pessimistic* (each row is judged by only ∼37% of the trees). It is a property of **bagging**, not unique to RF. **But** once you *tune* against OOB it is no longer unbiased - keep an untouched test set for the final reported number.

From Bagging to Random Forest

Bagged trees are too alike – force them to disagree.

The catch: bagged trees look alike

Bootstrap changes the *rows*, but every tree still sees **all the features**. So the **same strong feature** (Titanic: `sex`) tends to win the top split in nearly every tree.

Similar top splits $\Rightarrow$ similar trees $\Rightarrow$ **high** ρ $\Rightarrow$ the $\rho\sigma^2$ **floor stays high**. Averaging can only do so much.

To push the floor down we must make the trees **disagree more** – we must **decorrelate** them.

Random Forest: hide most features at each split

Random Forest (Breiman, 2001) = bagged trees where **each split** may only choose from a **random subset of F features**.

Predict-first: forcing each split to ignore most features – does that make the individual trees *better* or *worse*?

Random Forest: hide most features at each split

Random Forest (Breiman, 2001) = bagged trees where **each split** may only choose from a **random subset of F features**.

Predict-first: forcing each split to ignore most features – does that make the individual trees *better* or *worse*?

Each tree gets **slightly worse** (σ^2 up a bit). But the strong feature can no longer dominate every tree, so the trees **disagree more** (ρ down a lot).

The floor $\rho\sigma^2$ drops faster than σ^2 rises – so the **average wins**: trade a little per-tree accuracy for much less correlation.

How many features per split?

F is the decorrelation knob (sklearn: `max_features`). Rules of thumb:

- **Classification**: $F = \sqrt{P}$.
- **Regression**: $F = P/3$.
- smaller $F \Rightarrow$ more decorrelation (lower ρ), but weaker trees.

sklearn defaults, watch out: `RandomForestClassifier` uses `max_features="sqrt"` – it matches the rule. But `RandomForestRegressor` defaults to `max_features=1.0` (**all features**, since v1.1) – it does *not* apply the $P/3$ rule for you. On regression, lower it by hand.

Random Forest in Practice

How many trees, which knobs, and what it still cannot do.

Predict-first: do more trees eventually overfit?

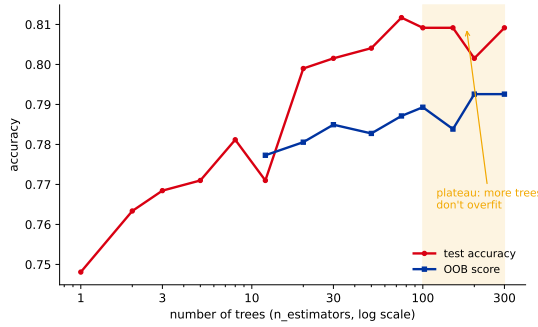
You can add hundreds of trees. Does test error eventually **rise** (overfit), like a too-deep single tree?

Predict-first: do more trees eventually overfit?

You can add hundreds of trees. Does test error eventually **rise** (overfit), like a too-deep single tree?

No. It **drops then plateaus**. More trees only shrink the $(1-\rho)/M$ term – they cannot overfit.

Precise: the *number of trees* does not overfit (the average just converges). *Deeper* trees / noisy features still can. Pick M "big enough, then stop" – you do not tune it by CV.



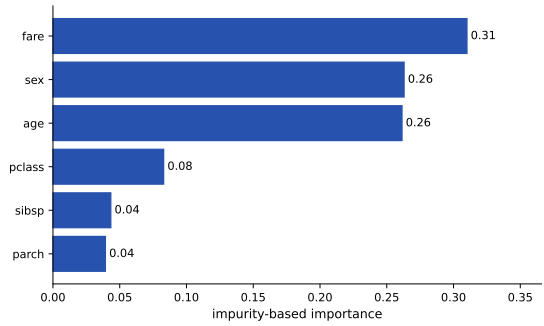
Titanic: rises, then flat past ∼100 trees.

Contrast (foreshadow [19]): in **boosting**, more trees *can* overfit – there, trees are added to correct each other, not averaged independently.

Feature importance (the default version)

A forest sums, over all trees, how much each feature's splits **dropped impurity**: `rf.feature_importances_`. A quick read of what drove the model.

On this forest `fare` and `age` outrank `sex` – the *opposite* of [17]'s single tree, where `sex` dominated.



Read with care. Impurity importance is **biased toward high-cardinality** features – continuous `fare` / `age` have many split points, so they soak up importance. **Permutation importance** and **SHAP** (the trustworthy versions) get their own lecture later.

Random forests in sklearn

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
        n_estimators=300, max_features="sqrt",
        oob_score=True, n_jobs=-1, random_state=509)
rf.fit(X_train, y_train)      # Titanic
print(rf.oob_score_)          # free validation estimate
```

Like a single tree: **no scaling needed** (callback: preprocessing), but sklearn still needs **numeric input** – encode categoricals yourself. Use `RandomForestRegressor` for the Yerevan rent target.

Key knobs

hyperparameter	what it does
`n_estimators`	more trees = steadier, then plateaus; costs compute. "Big enough, then stop."
`max_features`	**the decorrelation knob** (F). Lower = more diverse trees.
`max_depth` / `min_samples_leaf`	per-tree complexity (usually leave the trees deep).
`oob_score`	free held-out estimate (`=True`).
`n_jobs`	trees are independent $\Rightarrow$ trivially parallel (`=-1` uses all cores).
`class_weight`	`"balanced"` for class imbalance (callback [17]).

Extra Trees (`ExtraTreesClassifier`): picks split thresholds at *random* and, by default, **does not bootstrap** – even more randomization, faster, sometimes lower variance.

Strengths and weaknesses

Strengths

- Strong, low-tuning **default** model.
- Robust; mixed feature types, no scaling.
- **OOB** = built-in validation.
- Trees independent $\Rightarrow$ trivially **parallel**.

Weaknesses

- Less **interpretable** than one tree.
- Memory / inference cost (hundreds of trees).
- **Inherited from trees**: still axis-aligned boxes ("averaging boxes gives boxes"); extrapolates poorly.

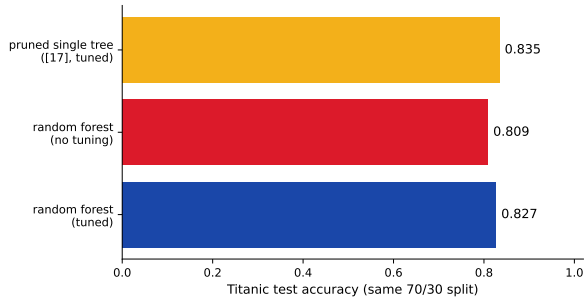
Bonus – proximities: how often two rows land in the *same leaf* across the forest is a similarity measure $\rightarrow$ outlier detection, missing-value imputation.

Reality check: did the forest beat one tree?

The chapter's running Titanic score:

- pruned single tree ([17]): **0.835**
- random forest, untuned: **0.809**
- random forest, CV-tuned: **0.827**

Ensembling was supposed to win. On *this* data it does not quite - and that is a lesson, not a bug.



Why the tree keeps up: a small set (1309 rows) with **one dominant feature** (`sex`) is already captured by a compact pruned tree. The comparison even flatters it - the tree's 0.835 is the *best* a pruning search found (picked on test), while the RF's 0.809 is **zero tuning**. A forest's edge shows with **more rows, more features, noisier signal**, and **far less tuning effort** - and recall it hid `sex` at many splits, which is exactly why `fare` / age rose in the importance plot.

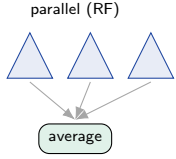
Bridge: two ways to combine trees

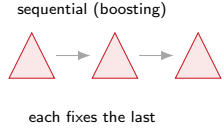
Random Forest (this deck)

- Trees grown **in parallel**, independently.
- Each sees resampled rows + a feature subset.
- **Averaging** cuts **variance**.

Boosting ([19])

- Trees grown **in sequence**.
- Each fixes the previous ones' **mistakes**.
- Sequential correction cuts **bias**.





Next ([19]): gradient boosting – and why *there*, unlike here, more trees *can* overfit.

Recap

- A single tree is **high variance**; **bagging** averages many trees on bootstrap samples to cut variance (not bias).
- Averaging cuts variance toward a **floor** set by how *correlated* the trees are; more trees alone cannot beat that floor.
- **Random Forest** = bagging + a **random feature subset per split** to lower ρ and drop that floor.
- **OOB** gives free validation; more trees **never overfit**.

Workflow: a Random Forest (`n_estimators` $\approx$ 300–500, `oob_score=True`) as a strong baseline $\rightarrow$ tune `max_features` (and depth if needed) by CV $\rightarrow$ read importances with care $\rightarrow$ reach for **boosting** ([19]) when you need more.

HW10 – bag the tree

1. **Titanic (chapter project):** fit a `RandomForestClassifier`; report the **OOB score** and compare it to 5-fold **CV**; compare **accuracy** to your L09 single tree (the chapter's running score), and also report **F1 / ROC-AUC** (Titanic is imbalanced).
2. Sweep `n_estimators` and plot the **plateau**; sweep `max_features` and watch it matter.
3. Read `feature_importances_`; note any high-cardinality bias.
4. **Regression:** `RandomForestRegressor` on the Yerevan rent toy; plot error vs `n_estimators`.

Bonus: swap in `ExtraTreesClassifier` and compare; argue from the variance formula why averaging *cannot* beat the $\rho\sigma^2$ floor.